

COCO API : TEMPS RÉEL, GESTION DE QUEUE & NOTIFICATIONS

Spécifications d'implémentation NestJS 12, Socket.io, BullMQ & Notifications Omnicanal

PROJET & APPLICATION	EXPERTISE & RÔLE	DATE & STATUT
Coco API (CocoTaille / CocoMoussou) Plateforme Salons de Beauté (Abidjan / Côte d'Ivoire)	Lead Architecte & Expert NestJS Architecture Événementielle & Systèmes Distribués	30 Septembre 2026 VALIDÉ POUR TRANSMISSION

RÉSUMÉ EXÉCUTIF

Ce document technique est destiné à l'équipe d'ingénierie Coco API. Il fournit les directives d'architecture pour moderniser la plateforme vers un modèle réactif et événementiel. Il détaille la mise en œuvre du temps réel pour la file d'attente (écrans salon et smartphones clients), la gestion asynchrone des délais de grâce (résolution automatique des No-Show en 10 minutes), l'expiration automatique des créneaux non payés, ainsi qu'un système complet de notifications multi-canal adapté aux spécificités de la Côte d'Ivoire (WhatsApp, SMS, Push FCM, In-App).

TABLE DES MATIÈRES SYNTHÉTIQUE

- 01 Diagnostic de l'Existant & Vulnérabilités Identifiées**
Analyse du module Queue, synchronisme et absence de moteur de jobs.
- 02 Architecture Cible : Modèle Événementiel Réactif (DDD)**
Découplage strict entre Domain Events, WebSockets et Workers asynchrones.
- 03 Benchmark & Stack Technologique Recommandée**
Sélection argumentée de Socket.io, BullMQ, Redis, FCM et WhatsApp/SMS.
- 04 Cas d'Usage Clés & Mécanismes Métier Détaillés**
Cycle de vie d'un ticket, timer de grâce de 10 min, webhooks Mobile Money.
- 05 Évolution du Schéma de Données (Prisma)**
Modèle de persistance Notification et stratégie multi-canal.
- 06 Feuille de Route d'Implémentation en 4 Phases**
Plan d'exécution pas à pas avec bonnes pratiques pour l'équipe.

1. ÉTAT DES LIEUX DE LA CODEBASE & CONSTAT

Analyse du code source existant dans le projet coco_api

L'inspection de la solution coco_api met en évidence une application NestJS 12 et Prisma v6 très bien structurée selon les principes de la Clean Architecture (couches domain, application, infrastructure, presentation). Les entités, DTOs et interfaces de repository sont isolés et testables.

Constat des 3 Manques Techniques Majeurs :

- **1. Polling HTTP sur la Queue** : Le dashboard de file d'attente (GetLiveQueueDashboardUseCase) est uniquement accessible via des requêtes REST répétées. En l'absence de communication bidirectionnelle, les écrans TV des salons et les applications mobiles doivent interroger le serveur en continu, ce qui sature la bande passante et induit une latence dans l'affichage des numéros appelés.
- **2. Non-Résolution des Délais de Grâce (No-Show)** : Lorsqu'un ticket est appelé dans QueueLifecycleUseCase, un délai de 10 minutes est assigné dans callDeadlineAt. Toutefois, aucun processus autonome ne vient clore le ticket si le client ne se présente pas. Le ticket reste indéfiniment en statut CALLED, bloquant la chaîne opérationnelle du salon.
- **3. Traitements Tiers Bloquants** : L'envoi de notifications (SMS, WhatsApp, Push) au sein du cycle de vie des requêtes HTTP expose le serveur aux latences des API externes (Orange, MTN, Wave, Meta, Twilio), avec un risque élevé de timeouts et de saturation de l'Event Loop Node.js.

DIRECTIVE LEAD ARCHITECTE

Le métier applicatif ne doit jamais être ralenti par des opérations d'I/O externes. Les Use Cases doivent exécuter leur logique pure, sauvegarder en base, puis émettre un événement. L'infrastructure se charge de réagir en temps réel ou de manière différée.

2. ARCHITECTURE CIBLE : DÉCOUPLAGE ÉVÉNEMENTIEL (DDD)

Synergie entre Domain Events, WebSockets et Distributed Queues

L'architecture cible adoptée s'articule autour du principe d'Event-Driven Architecture (EDA) :

- **Couche Application (Métier Pur)** : Les usecases (ex: QueueLifecycleUseCase) modifient l'état en base via les repositories Prisma, puis publient un événement domaine typé via EventEmitter2 (@nestjs/event-emitter). Le use case ignore totalement si l'événement sera diffusé sur une TV ou envoyé par SMS.
- **Couche Présentation Temps Réel (WebSockets)** : Une Gateway Socket.io dédiée intercepte les événements (@OnEvent) et pousse la donnée dans les Rooms ciblées : la room du salon (salon:\${salonId}) pour l'affichage général, et la room client (customer:\${customerId}) pour le smartphone personnel.
- **Couche Infrastructure Asynchrone (BullMQ)** : Un Event Listener spécifique intercepte également l'événement pour déléguer les actions lourdes à des workers Redis : programmation d'un timer différé de 10 min pour le contrôle No-Show, et envoi résilient des SMS / WhatsApp.

3. BENCHMARK & SÉLECTION DES TECHNOLOGIES

Choix des briques modernes pour une haute résilience

Les technologies retenues garantissent une compatibilité maximale avec l'écosystème NestJS 12 tout en tenant compte de l'infrastructure mobile et réseau en Côte d'Ivoire (réseaux 3G/4G hétérogènes, usage massif de WhatsApp et du SMS).

Périmètre	Solution Retenue	Alternative Rejetée	Avantages & Justification Technique
Temps Réel (WebSockets)	Socket.io + Redis Adapter (@nestjs/platform-socket.io)	WS natif pur Server-Sent Events (SSE)	- Support natif des Rooms et Namespaces par salon. - Reconnexion automatique avec backoff. - Fallback HTTP Long-Polling vital pour les mobiles en zone réseau faible.
Files Asynchrones & Délais	BullMQ + Redis (@nestjs/bullmq, ioredis)	RabbitMQ / Kafka Simple @Cron NestJS	- Support des Delayed Jobs (indispensable pour le timer No-Show 10 min). - Gestion native des retries exponentiels. - Léger, persistant et hyper-performant sur Redis.
Supervision des Queues	@bull-board/nestjs	Scripts CLI / Logs bruts	- Interface web d'administration pour observer l'état des jobs (waiting, active, failed). - Relance des jobs échoués (Dead-Letter Queue) en 1 clic.
Bus Événements Métier	@nestjs/event-emitter (EventEmitter2)	Couplage direct dans UseCase	- Découplage complet de la Clean Architecture. - Permet de brancher WebSockets, BullMQ et Analytics sans toucher au Use Case.
Push Notifications	Firebase Cloud Messaging (FCM v1)	OneSignal propriétaire	- Gratuit, standard mondial Android & iOS. - Envoi de payloads de données silencieux pour mise à jour de l'app en arrière-plan.
SMS & WhatsApp (Afrique)	WhatsApp Cloud API + Termii (Fallback: Twilio / Infobip)	Passerelles locales non documentées	- WhatsApp : taux d'ouverture > 95% en Côte d'Ivoire. - Termii : excellent taux de délivrabilité SMS sur les préfixes Orange/MTN/Moov CI.

Pourquoi éliminer le Cron PostgreSQL au profit de BullMQ ?

L'approche classique par Cron (@Cron de NestJS exécuté toutes les minutes) impose d'interroger la table QueueTicket avec `SELECT * FROM ... WHERE callDeadlineAt < NOW()`. Cela provoque une surcharge d'I/O et de verrouillages sur PostgreSQL, tout en créant une latence pouvant aller jusqu'à 60 secondes. Avec BullMQ, le timer est différé dans Redis : l'exécution est déclenchée à la seconde exacte sans solliciter la base PostgreSQL durant toute la période d'attente.

4. CAS D'USAGE CLÉS & FLUX OPÉRATIONNELS

Application concrète au cycle de vie de la file et des réservations

A. Scénario : Appel d'un Ticket & Résolution Automatique du No-Show

Lorsqu'un coiffeur appelle un ticket via `QueueLifecycleUseCase.callTicket()` :

- **1. Changement d'état** : Le ticket passe en CALLED, avec `callDeadlineAt = NOW() + 10 minutes`.
- **2. Émission de l'événement** : Le usecase émet l'événement `QueueTicketCalledEvent` avec les métadonnées.
- **3. Diffusion Temps Réel** : La Gateway `Socket.io` diffuse sur salon: `{salonId}` (mise à jour de l'écran TV d'affichage avec carillon sonore) et sur customer: `{customerId}` (vibration et compte à rebours interactif de 10 min sur le smartphone du client).
- **4. Armement du Timer BullMQ** : Un job différé `check-ticket-no-show` est inséré dans Redis avec delay: 600000 (10 min).
- **5. Traitement à Échéance** : À T+10 min, si le ticket est toujours en statut CALLED (le client ne s'est pas présenté), le worker le passe en NO_SHOW, libère la place et avertit le salon pour appeler le ticket suivant.

```
// Worker BullMQ dédié à l'automatisation du No-Show (src/module/queue/infrastructure/workers/)
@Processor('queue-lifecycle')
export class QueueLifecycleWorker extends WorkerHost {
  constructor(
    @Inject(QueueRepository) private readonly queueRepo: IQueueRepository,
    private readonly eventEmitter: EventEmitter2,
  ) { super(); }

  async process(job: Job<{ ticketId: string; salonId: string }>) {
    if (job.name === 'check-ticket-no-show') {
      const ticket = await this.queueRepo.findById(job.data.ticketId);
      if (ticket && ticket.status === QueueTicketStatus.CALLED) {
        ticket.markAsNoShow();
        await this.queueRepo.update(ticket);
        this.eventEmitter.emit('queue.ticket.no_show', new QueueTicketNoShowEvent(ticket));
      }
    }
  }
}
```

B. Scénario : Verrouillage Temporaire de Créneau (Hold 15 min)

Lors de la prise de RDV avec acompte Mobile Money, la réservation est en attente (PENDING_DEPOSIT). Un job différé `release-unpaid-booking` est planifié à 15 minutes. Si aucun webhook de paiement réussi n'est reçu, le créneau horaire est immédiatement restitué à la disponibilité du salon.

C. Scénario : Confirmation Instantanée des Paiements Webhooks

Dès validation du webhook Mobile Money (Wave, Orange, MTN), l'événement `payment.succeeded` est envoyé sur la socket du client. L'écran de l'application mobile affiche instantanément le ticket validé et le reçu numérique sans manipulation supplémentaire.

5. ÉVOLUTION DU SCHÉMA PRISMA & NOTIFICATIONS

Persistence de l'historique In-App et Architecture Multi-Fournisseurs

Dans le schéma prisma actuel, la table NotificationPreference configure les canaux acceptés par l'utilisateur, mais il manque la table matérialisant les notifications reçues. L'ajout du modèle Notification ci-dessous comble ce manque pour alimenter le centre d'alertes in-app :

```
// À intégrer dans prisma/schema.prisma (Module Notifications)
model Notification {
  id          String           @id @default(uuid())
  userId      String
  salonId     String?          // Salon expéditeur si applicable
  title       String
  body        String
  channel     NotificationChannel // PUSH, SMS, IN_APP, WHATSAPP
  isRead      Boolean           @default(false)
  readAt      DateTime?
  data        Json?             // { ticketId: "...", bookingId: "...", actionUrl: "... }
  createdAt   DateTime           @default(now())

  user        User              @relation(fields: [userId], references: [id], onDelete: Cascade)
  salon       Salon?            @relation(fields: [salonId], references: [id], onDelete: SetNull)

  @@index([userId, isRead])
  @@index([userId, createdAt])
}
```

Design Pattern Strategy pour le NotificationModule

Le service de notification orchestre l'envoi en interrogeant la fabrique de fournisseurs selon les préférences de l'utilisateur :

- **Canal In-App** : Insertion de l'enregistrement dans la table Notification et émission WebSocket instantanée.
- **Canal Push (Firebase FCM)** : Envoi vers les tokens FCM enregistrés lors de la connexion mobile.
- **Canal WhatsApp** : Transmission d'un message interactif via l'API officielle WhatsApp Business (haute priorité pour les rappels de rdv et alertes de ticket appelé).
- **Canal SMS (Termii / Twilio)** : Envoi du SMS transactionnel si le client est sans connexion mobile data.

ROBUSTESSE FACE AUX PANNES DES OPÉRATEURS

Chaque tentative d'envoi de SMS ou WhatsApp est confiée à un worker BullMQ avec 3 tentatives automatiques et backoff exponentiel. En cas d'indisponibilité temporaire d'un opérateur télécom, le message n'est jamais perdu.

6. FEUILLE DE ROUTE D'IMPLÉMENTATION EN 4 PHASES

Plan d'exécution structuré pour l'équipe de développement

Phase	Intitulé & Scope	Charge Estimée	Livrables Techniques
Phase 1	Socle Infra & Message Broker (Redis + BullMQ + Event-Emitter)	2 - 3 jours	• Déploiement Redis (ex: Railway Redis) • Configuration @nestjs/bullmq & @nestjs/event-emitter • Montage de l'UI d'administration Bull-Board
Phase 2	Temps Réel & Gateway WebSockets (Socket.io)	3 - 4 jours	• QueueGateway avec Rooms dynamiques par salon et par client • Authentification JWT sur handshake WebSocket • Événements ticket:called, ticket:waiting, ticket:done
Phase 3	Automatisation du Cycle de Vie (Workers No-Show & Holds)	2 - 3 jours	• Worker check-ticket-no-show avec délai de grâce de 10 min • Worker release-unpaid-booking (délai 15 min) • Synchronisation automatique avec Booking et CRM
Phase 4	Module Notifications Omnicanal (In-App, Push, WhatsApp, SMS)	4 - 5 jours	• Migration Prisma de la table Notification • Intégration Firebase Admin SDK (FCM) • Intégration WhatsApp Cloud API & Termii SMS Provider

Règles d'Ingénierie pour l'Équipe

- **Idempotence des Jobs** : Chaque tâche BullMQ doit avoir un jobId explicite (ex: no-show-\${ticketId}) pour interdire les doublons en cas de double clic.
- **Typage Strict des Payloads** : Créer un fichier domain/events/queue-ticket.events.ts contenant toutes les classes d'événements typées.
- **Monitoring & Alerting** : Brancher des alertes en cas de jobs envoyés dans la Dead-Letter Queue (DLQ).

SYNTHÈSE DE LA DIRECTION TECHNIQUE

Cette modernisation confère à Coco API une infrastructure digne des meilleurs standards SaaS internationaux. Elle supprime la latence en salon, automatise les opérations chronophages et garantit une expérience utilisateur fluide et moderne.